Pillars of Object-Oriented Programming (OOP)

1. Abstraction

- Abstraction means showing what an object can do while hiding how it does it.
- When driving a car, we use the steering wheel, accelerator, and brakes without needing to understand how the engine works internally.
- Similarly, in programming, we can call accelerate() without knowing its internal implementation.

Abstraction in Java

- Java supports abstraction through **interfaces** and **abstract classes**.

```
interface Car {
   void startEngine();
   void accelerate();
   void brake();
}
```

- The Car interface defines what operations a car must provide, but not how they work.
- A class implements the interface and provides the actual code.

```
class SportsCar implements Car {
   public void startEngine() {
      System.out.println("Engine started");
   }
   public void accelerate() {
      System.out.println("Car is accelerating");
   }
   public void brake() {
      System.out.println("Car is braking");
   }
}
```

- We can use the interface without depending directly on the implementation:

```
Car myCar = new SportsCar();
myCar.startEngine();
myCar.accelerate();
```

Benefits

- Hides unnecessary implementation details.
- Makes code easier to use and understand.
- Allows different classes to provide their own implementations of the same operations.
- Makes it easier to change internal code without changing the code that uses it, provided the interface stays the same.

2. Encapsulation

- Encapsulation means keeping data and the methods that work on that data together in a class, while controlling access to them.
- A car's speed should not be changed directly to an unrealistic value. The car should provide methods to control its speed.

Encapsulation in Java

```
class SportsCar {
   private int currentSpeed = 0;
   public int getSpeed() {
      return currentSpeed;
   }
   public void accelerate() {
      currentSpeed += 20;
   }
   public void brake() {
      currentSpeed -= 20;
      if (currentSpeed < 0) {
         currentSpeed = 0;
      }
   }
}
```

Understanding the code

- private int currentSpeed: Prevents other classes from directly accessing the speed field.
- getSpeed(): Allows other classes to read the speed.
- accelerate(): Increases the speed through a controlled method.
- brake(): Decreases the speed without allowing it to become negative.

Access Modifiers in Java

Modifier	Meaning
public	Accessible wherever the declaring type and member are accessible.
private	Accessible only within the declaring class.
protected	Accessible within the same package and through inheritance, subject to Java's access rules.
Default (no modifier)	Accessible within the same package.

Getters and Setters

- Getter**:** Retrieves a field's value.
- Setter**:** Updates a field's value.

A setter can validate input before changing the field. Getters and setters are not required for every field; expose only what is needed.

Benefits

- Prevents uncontrolled access to data.
- Allows validation before data is changed.
- Makes internal changes easier to manage.
- Keeps related data and behaviour together.

3. Inheritance

- Inheritance allows a class to acquire accessible fields and methods from another class, so common code can be reused.
- The existing class is called the parent class or superclass. The new class is called the child class or subclass.

Example

- Different types of cars share common features such as a brand, model, engine, and speed. However, a manual car has gears, while an electric car has a battery.
- Instead of writing all common features separately in both classes, we can define them in a parent class called Car.

Inheritance in Java

```
class Car {
    protected String brand;
    protected int currentSpeed = 0;
    public Car(String brand) {
        this.brand = brand;
    }
    public void startEngine() {
        System.out.println("Engine started");
    }
    public void brake() {
        currentSpeed = Math.max(0, currentSpeed - 20);
    }
}
class ManualCar extends Car {
    private int currentGear = 0;
    public ManualCar(String brand) {
        super(brand);
    }
    public void shiftGear(int gear) {
        currentGear = gear;
    }
}
class ElectricCar extends Car {
    private int batteryLevel = 100;
    public ElectricCar(String brand) {
        super(brand);
    }
    public void chargeBattery() {
        batteryLevel = 100;    }   }
```

Understanding the code

- Car contains features common to different cars.
- ManualCar extends Car inherits accessible features from Car.
- ElectricCar extends Car also inherits accessible features from Car.
- ManualCar adds currentGear and shiftGear().
- ElectricCar adds batteryLevel and chargeBattery().
- super(brand) calls the parent class constructor.

Both child classes can use the inherited startEngine() and brake() methods without rewriting them.

Benefits

- Reuses common code.
- Reduces duplication.
- Allows child classes to add specialized features.
- Represents an **is-a** relationship, such as a ManualCar being a type of Car.

Note**:** Private members cannot be accessed directly by child classes. Use inheritance when the parent-child relationship makes sense, not merely to reuse any available code.

4. Polymorphism

- Polymorphism means that the same method name or interface can produce different behaviour depending on the arguments or the actual object.
- The word means "many forms."
- Polymorphism has two common forms: compile-time and runtime polymorphism.

4.1 Compile-Time Polymorphism (Method Overloading)

- Method overloading means defining multiple methods with the same name but different parameter lists.
- The compiler determines which method to call based on the arguments.

```
class Car {
   public void accelerate() {
      System.out.println("Normal acceleration");
   }
   public void accelerate(int speed) {
      System.out.println("Accelerating by " + speed);
   }
}
```

Both methods are named accelerate(), but their parameter lists differ.

```
Car car = new Car();
car.accelerate();
car.accelerate(50);
```

The first call selects the method without parameters. The second selects the method with an integer parameter.

Important: Changing only the return type does not create a valid overload.

4.2 Runtime Polymorphism (Method Overriding)

- Method overriding occurs when a child class provides its own implementation of an inherited method with the same method signature.
- The method that runs is determined by the actual object's class at runtime.

```
class Car {
    public void accelerate() {
        System.out.println("Car is accelerating");
    }
}
class ManualCar extends Car {
    @Override
    public void accelerate() {
        System.out.println("Manual car is accelerating");
    }
}
class ElectricCar extends Car {
    @Override
    public void accelerate() {
        System.out.println("Electric car is accelerating");
    }
}
```

Using runtime polymorphism

```
Car car1 = new ManualCar();

Car car2 = new ElectricCar();

car1.accelerate();

car2.accelerate();
```

Output:

```
Manual car is accelerating

Electric car is accelerating
```

Both variables have the reference type Car, but each call runs the implementation belonging to the actual object.

Benefits

- Allows different classes to provide their own behaviour.
- Makes code easier to extend with new implementations.
- Allows common code to work with different object types through a shared parent class or interface.

5. Abstraction vs Encapsulation

Abstraction	**Encapsulation**
Hides implementation details.	Controls access to internal data and behaviour.
Focuses on what an object does.	Focuses on how its internal state is managed and protected.
Commonly uses interfaces and abstract classes.	Commonly uses classes and access modifiers such as private.
Example: Calling accelerate() without knowing how it works.	Example: Preventing direct changes to currentSpeed.

6. Inheritance vs Polymorphism

Inheritance	Polymorphism
Allows a class to inherit accessible features from another class.	Allows the same operation to behave differently.
Helps reuse common code.	Helps provide different implementations through a shared method or interface.
Example: ElectricCar extends Car.	Example: ElectricCar overrides accelerate().

7. Quick Revision

- Abstraction: Shows what an object can do while hiding how it works.
- Encapsulation: Groups data and methods together and controls access to them.
- Inheritance: Allows a child class to reuse accessible features of a parent class.
- Polymorphism: Allows the same operation to behave differently depending on the arguments or actual object.
- Overloading: Same method name, different parameter lists; resolved at compile time.
- Overriding: A child class provides a new implementation of an inherited method; runtime dispatch selects the implementation for the actual object.

Remember: Abstraction hides implementation details, encapsulation controls access, inheritance reuses common features, and polymorphism allows different behaviours through a shared operation.